```java
package com.ravi.hash_set_demo;

//add, delete, display and exit

import java.util.HashSet;
import java.util.Scanner;

public class HashSetDemo4
{
    public static void main(String[] args)
    {
  HashSet<String> hashSet = new HashSet<>();

  while (true)
    {
     System.out.println("Options:");
     System.out.println("1. Add element");
     System.out.println("2. Delete element");
     System.out.println("3. Display HashSet");
     System.out.println("4. Exit");

     System.out.print("Enter your choice (1/2/3/4): ");
     int choice = Integer.parseInt(IO.readln());

     switch (choice)
        {
     case 1 ->
     {
        String elementToAdd = IO.readln("Enter the element to add :");
        if(hashSet.add(elementToAdd))
        {
            IO.println("Element "+elementToAdd+" added successfully..");
        }
        else
        {
            IO.println(elementToAdd+" element is already available..");
        }
     }
     case 2->
     {
        String elementToDelete = IO.readln("Enter the element to delete :");
        if(hashSet.isEmpty())
        {
            IO.println("HashSet is empty, Nothing to delete");
            System.exit(0);
        }

        boolean isRemoved = hashSet.remove(elementToDelete);

        if(isRemoved)
        {
            IO.println(elementToDelete+" is deleted successfully");
        }
        else
        {
            IO.println(elementToDelete+" is not available ");
        }
     }
     case 3 ->
     {
       IO.println(hashSet);
     }

     case 4->
     {
      System.out.println("Exiting the program.");
       System.exit(0);
     }
     default ->
     System.out.println("Invalid choice. Please try again.");
     }

     System.out.println();
    }
  }
}
```

```java
package com.ravi.hash_set_demo;

import java.util.HashSet;
import java.util.Hashtable;

public class HashSetDemo5
{
    public static void main(String[] args)
    {

        HashSet<String> hs1 = new HashSet<String>();
        hs1.add("Java");
        hs1.add(new String("Java"));
        IO.println("hs1 size is :"+hs1.size());

        HashSet<String> hs2 = new HashSet<String>();
        hs2.add(new String("Java"));
        hs2.add(new String("Java"));
        IO.println("hs2 size is :"+hs2.size());

        HashSet<StringBuffer> hs3 = new HashSet<StringBuffer>();
        hs3.add(new StringBuffer("Java"));
        hs3.add(new StringBuffer("Java"));
        IO.println("hs3 size is :"+hs3.size()); //2

    }
}
```

Map<K,V> interface :

* It is a predefined interface available in java.util package from JDK 1.2V.

* It works with group of objects i.e. **key and value pair. {key=value}**

* It is a separate interface, It does not have any concern with **Collection interface** hence, It will not support Collection interface method.

* It stores everything in the form of key & value.

* Here key and value both are objects.

* Key must be unique **(Duplicate keys are not allowed)** but value may be duplicate.

* If we try to insert duplicate key then old key value will be replaced by new key value.

* Each combination of **key and value pair is known as Entry.**

* Actually Entry<K,V> is an interface which is defined inside Map<K,V> interface.

```java
public interface Map<K,V>
{
    public interface Entry<K,V>
    {
        //key & value pair
    }
}
```

In .java file -> Map.Entry
In .class file -> Map$Entry

Diagram :

key	value
ravi@gmail.com	ravi$%^&
scott@gmail.com	scott*&12
alen@gmail.com	alen#fgj^7

Entry (Combination of key & value pair)

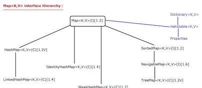
Here We have total 3 entries are available

* Before Map<K,V>, We have abstract class called Dictionary<K,V> and to extend this class we have another concrete class called Hashtable<K,V> in JDK 1.0

* We can apply iterator() method directly on Map interface but if we convert the map to set interface (entrySet()) then we can apply itrerator() method.

* We can fetch the key & value both objects by using forEach(BiConsumer<T,U> cons) method, forEach() is added as a default method inside Map interface from JDK 1.8v

Map<K,V> interface Hierarchy :



Methods of Map<K,V> interface :

1) V put(K key, V value) :- To insert one entry in the Map collection. It will return the value of old Object key, if the key is already available(Duplicate key), If key is not available (new key) then it will return null.

2)default V putIfAbsent(Object key, Object value) :- It will insert an entry, if and only if, key is not available , if the key is already available then it will not insert the Entry to the Map Collection.

3) V get(Object key) :- It will return corresponding key value, if the key is not available then it will return null.

4) Object getOrDefault(Object key, Object defaultValue) :- To avoid null value this method has been introduced from JDK 1.8V, here we can pass some defaultValue to avoid the null value.

5) boolean containsKey(Object key) :- To Search a particular key